

IdeaForge AI - Lesson 1: Backend Basics

Learning Path

HTML → CSS → JavaScript → React → Node.js → Express → MongoDB → REST APIs → AI
API → File Upload → PDF Reading → Authentication → Deploy

How the App Works

User uploads an idea/PDF → React Frontend → Express Backend → PDF Reader →
Gemini AI → MongoDB (optional) → Results shown in the frontend.

First Express Server

```
const express = require("express");  
const app = express();
```

```
app.get("/", (req, res) => {  
  res.send("Hello IdeaForge AI");  
});
```

```
app.listen(3000, () => {  
  console.log("Server Running");  
});
```

Explanation

1. `require("express")` imports Express.
2. `express()` creates the web server.
3. `app.get("/")` creates a route.
4. `req` contains data sent by the user.
5. `res` sends data back to the user.
6. `res.send()` returns plain text.
7. `app.listen(3000)` starts the server.

Receiving JSON

```
app.use(express.json());
```

```
app.post("/idea", (req, res) => {  
  console.log(req.body);  
});
```

The frontend sends JSON like { idea: "AI Attendance" }. Access it using req.body.idea.

Sending JSON Response

```
res.json({  
  score: 9,  
  market: "High"  
});
```

Fake AI Endpoint

```
app.post("/analyze",(req,res)=>{  
  const idea = req.body.idea;  
  const result = {  
    score: 8.5,  
    market: "High",  
    swot: "Strong innovation"  
  };  
  res.json(result);  
});
```

Gemini Prompt

```
const prompt = `  
Analyze this startup idea:  
  
${idea}  
`;
```

Never expose your AI API key in the frontend. Always call the AI from the backend.

Mini Challenge

Create a POST endpoint /student that receives name, department, and year, then returns a welcome message as JSON.